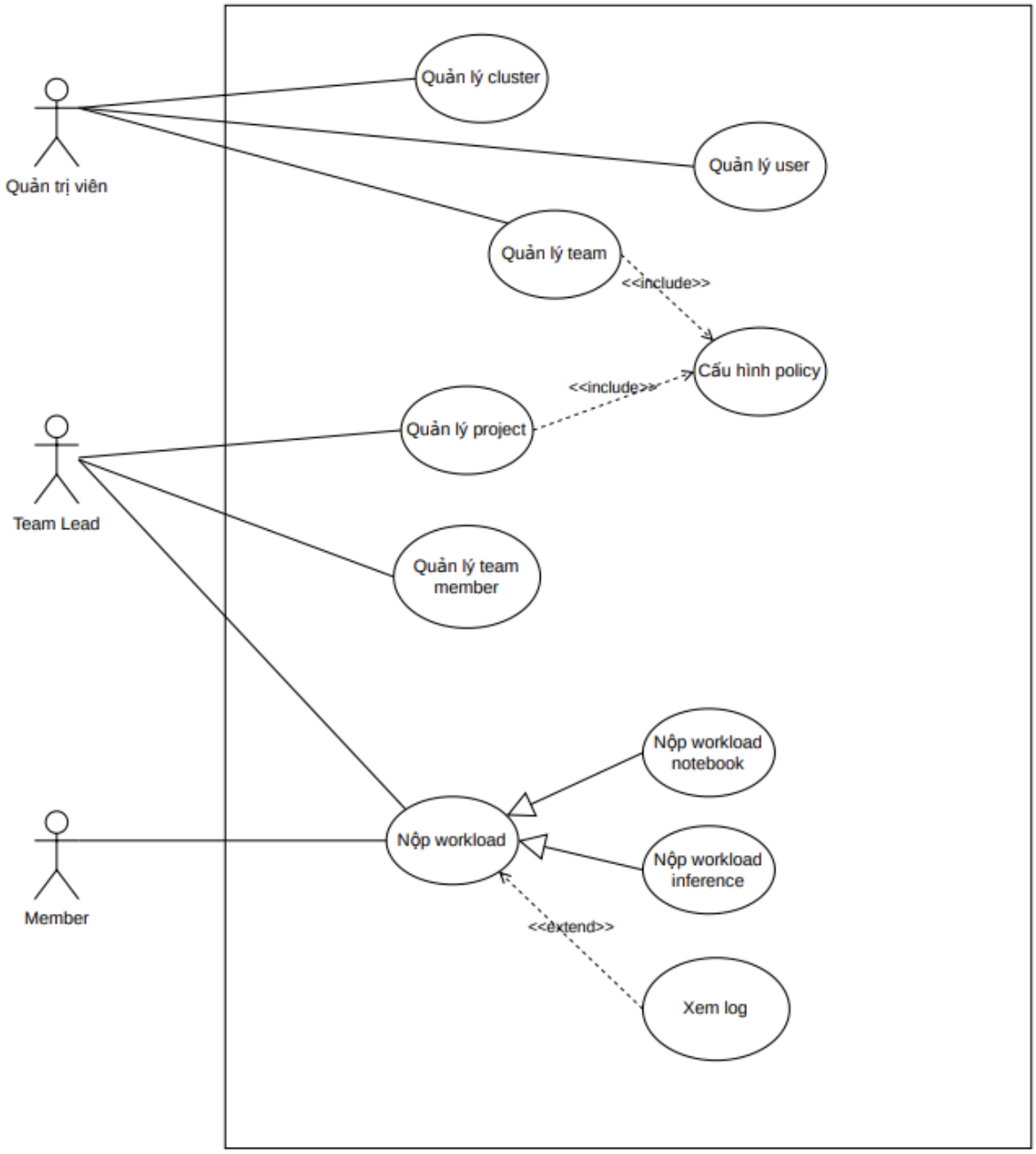
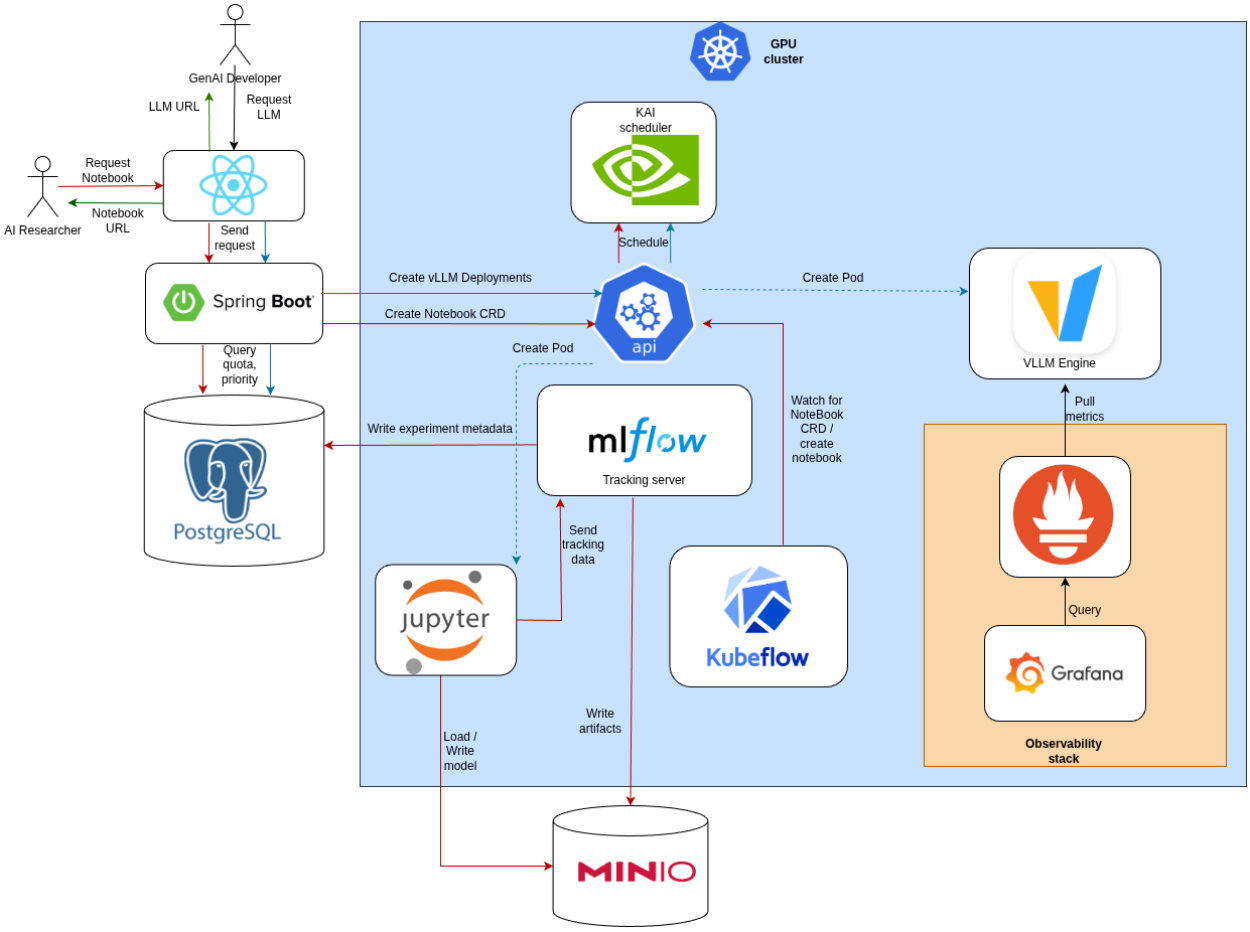
Tài liệu thiết kế các tính năng cho dự án GPU Hub

Tổng quan hệ thống

Biểu đồ Use Case:



Kiến trúc Tổng thể



GPU Hub được xây dựng theo kiến trúc phân lớp, theo hướng microservice. Giao diện người dùng React (SPA) giao tiếp với backend Spring Boot thông qua REST API. Backend tích hợp với một hoặc nhiều cụm Kubernetes GPU thông qua Kubernetes API server, điều phối workload bằng KAI Scheduler cho lập lịch GPU, Kubeflow cho quản lý vòng đời notebook, và các Deployment vLLM cho suy luận LLM.

Trạng thái bền vững được lưu trữ trong PostgreSQL. Artifact mô hình và dữ liệu thí nghiệm được lưu trong MinIO và được theo dõi qua MLflow Tracking Server. Khả năng quan sát hệ thống được cung cấp bởi bộ công cụ Prometheus + Grafana, thu thập metrics từ các pod đang chạy.

Bộ Công nghệ Sử dụng

Tầng	Công nghệ	Vai trò
Giao diện người dùng	React (SPA)	Giao diện cho tất cả các vai trò
Backend / API	Spring Boot 3 (Java)	Xử lý nghiệp vụ, REST API, điều phối Kubernetes
Cơ sở dữ liệu	PostgreSQL 15	Người dùng, token, team, project, policy, metadata workload

Lập lịch	KAI Scheduler (NVIDIA)	Xếp hàng ưu tiên có nhận thức GPU trên Kubernetes
Notebook Engine	Kubeflow Notebook Controller	Quản lý vòng đời Jupyter Notebook CRD
Inference Engine	vLLM	Phục vụ LLM thông lượng cao qua Kubernetes Deployment
Theo dõi thí nghiệm	MLflow Tracking Server	Metadata thí nghiệm và tham chiếu artifact
Lưu trữ đối tượng	MinIO	Trọng số mô hình, dataset artifact, đầu ra notebook
Quan sát hệ thống	Prometheus + Grafana	Thu thập metrics và hiển thị dashboard GPU/CPU/bộ nhớ
Xác thực	JWT + RBAC (Spring Security)	Xác thực phi trạng thái; refresh token lưu trong PostgreSQL
Điều phối container	Kubernetes (GPU cluster)	Quản lý node và tài nguyên

Xác thực và phân quyền:

Cơ chế xác thực:

Quá trình xác thực được thực hiện thông qua thông tin đăng nhập username/password. Sau khi xác thực thành công, máy chủ sẽ cấp một cặp mã thông báo:

Access Token (JWT): có thời hạn ngắn, được mang trong tiêu đề *Authorization*: Bearer của các API request tiếp theo.

Refresh Token: có thời hạn dài, được lưu trữ trên máy chủ trong PostgreSQL dựa trên bản ghi người dùng và được trả về cho máy khách dưới dạng cookie

Việc xoay vòng token tuân theo chuẩn OAuth 2.0. Khi access token hết hạn, client sẽ gửi refresh token đến endpoint auth/refresh. Máy chủ sẽ xác thực token dựa trên bản ghi cơ sở dữ liệu, cấp access token mới và tùy chọn xoay vòng refresh token.

Mô hình phân quyền:

Việc xác thực được thực hiện thông qua Role-based Access Control (RBAC) được triển khai trong Spring Security. Mỗi API endpoint được bảo vệ bởi các annotation ở cấp phương thức hoặc cấp bộ lọc để kiểm tra vai trò của người dùng đã được xác thực.

Một người dùng có thể thuộc nhiều team cùng lúc, giữ các vai trò khác nhau (MEMBER hoặc TEAM LEAD) trong mỗi team. Quyền hạn hiệu quả đối với các thao tác trong phạm vi team được suy ra từ role mà người dùng nắm giữ trong team cụ thể đang được truy cập.

Ma trận quyền hạn:

Operation	ADMIN	TEAM_LEAD	MEMBER
Quản lý clusters (add/remove/configure)	✓	✗	✗
Cấu hình policies	✓	✗	✗
Thêm / xóa teams	✓	✗	✗
Thêm / xóa projects	✓	✓ (own team)	✗
Thêm / sửa / xóa users	✓	✗	✗
Thêm user vào team	✓	✓ (own team)	✗
Xem tài nguyên cụm	✓	✓ (own team's clusters)	✗
Xem resource usage, quota, limit	✓	✓ (own team)	✗
Xem workload	✓	✓ (own team)	✗
Submit workload	✓	✓	✓
Xem log workload	✓	✓	✓
Dừng, xóa workload	✓	✓	✓

Tính năng Quản trị viên:

Quản lý các cluster:
Quản trị viên có thể đăng ký và cấu hình kết nối đến một hoặc nhiều cụm GPU Kubernetes. Mỗi cụm được xác định bằng endpoint của API server hoặc bằng cách tải lên tệp kubeconfig. Backend lưu trữ thông tin xác thực kết nối một cách an toàn và sử dụng chúng để giao tiếp với Kubernetes API server.

Quản lý policy:
Policy là một tập hợp các ràng buộc tài nguyên và tham số lập lịch mà quản trị viên sẽ phải cấu hình. Mỗi policy được liên kết với chính xác một cụm và được gắn với một team hoặc một project. Các policy tạo thành cơ sở của cây hàng đợi của KAI scheduler: các policy cấp team là con trực tiếp của nút lập lịch gốc, và các policy cấp project là con của nút policy của team quản lý project đó.

Quản lý project:
Project là các đơn vị con trong một team, thuộc về một cluster duy nhất có thể được tạo ra bởi ADMIN toàn cục hoặc TEAM LEAD của team. Mỗi project chỉ có duy nhất một policy liên kết. Người dùng gửi các workload thuộc một project, policy của project xác định hàng đợi lập lịch và các ràng buộc tài nguyên được áp dụng cho các workload đó.

Quản lý người dùng:
Quản trị viên có thể tạo và xóa người dùng nền tảng. Mỗi tài khoản người dùng có một tên người dùng duy nhất, mật khẩu được mã hóa và thông tin liên hệ. Mỗi người dùng sẽ có một role toàn cục là ADMIN hoặc USER, role với từng team tham gia là TEAM_LEAD hoặc MEMBER. Người dùng không được đăng ký tự động; tài khoản chỉ được cấp bởi quản trị viên.

Giám sát nền tảng:
Bảng điều khiển quản trị cung cấp cái nhìn tổng quan về trạng thái hoạt động của nền tảng:

- Các workload đang hoạt động trên tất cả các team và cluster (tên, chủ sở hữu, project, loại, trạng thái, mức sử dụng GPU/CPU/bộ nhớ, thời gian hoạt động)
- Mức tiêu thụ tài nguyên tổng hợp theo từng team và từng project so với hạn mức và giới hạn đã cấu hình
- Các khối lượng công việc đang chờ xử lý trong KAI Scheduler, với thời gian chờ ước tính

Tính năng của người dùng:

Submit workload:

Người dùng đã xác thực (MEMBER hoặc TEAM_LEAD) có thể gửi workload dưới bất kỳ project nào họ là thành viên. Quá trình gửi yêu cầu người dùng chọn project mục tiêu, loại workload, thông số tài nguyên và các tham số cấu hình riêng cho từng loại workload.

Các tham số gửi chung:

Tên workload	Định danh do người dùng đặt (phải duy nhất trong namespace của project)
Project	Project mục tiêu (xác định cluster, namespace và hàng đợi lập lịch)
Loại workload	NOTEBOOK hoặc LLM_INFERENCE
Số lượng GPU	Số GPU yêu cầu (số nguyên, ≥ 0)
Loại GPU	Model GPU cụ thể yêu cầu (tùy chọn; sử dụng node selector/label)
CPU request / limit	Tài nguyên CPU tính theo millicores
Memory request / limit	Tài nguyên bộ nhớ (ví dụ: '16Gi')
Độ ưu tiên	Độ ưu tiên lập lịch; bị giới hạn bởi max_priority trong policy của project

Workload Notebook:

Workload Notebook cấp phát một môi trường JupyterLab được hỗ trợ bởi pod có GPU. Backend tạo một custom resource (CRD) Kubeflow Notebook trong namespace Kubernetes của project mục tiêu. Kubeflow Notebook Controller theo dõi CRD này và tạo pod tương ứng.

Các tham số riêng cho workload này:

- Docker image: image container nền cho notebook (ví dụ: jupyter/tensorflow-notebook:latest hoặc image từ registry riêng)
- Kích thước PVC (Persistent Volume Claim) — dung lượng lưu trữ cho thư mục home của notebook

Khi pod notebook đã chạy, backend phân giải service trong cluster và hiển thị URL Notebook cho người dùng. URL được trả về qua API và hiển thị trong giao diện dưới dạng liên kết.

Container notebook được cấu hình sẵn biến môi trường MLFLOW_TRACKING_URI trỏ đến MLflow Tracking Server của nền tảng. Người dùng có thể ghi lại thí nghiệm, tham số, chỉ số và artifact ngay từ bên trong notebook. Dữ liệu artifact được lưu bền vững vào MinIO; metadata thí nghiệm được ghi vào cơ sở dữ liệu MLflow.

Workload LLM Inference:

Workload suy luận LLM triển khai một server suy luận vLLM dưới dạng Kubernetes Deployment trong namespace của project mục tiêu. Backend xây dựng manifest Deployment, áp dụng thông số tài nguyên của người dùng và cấu hình lệnh vllm serve, sau đó gửi lên Kubernetes API.

Các tham số riêng:

- Đường dẫn mô hình / HuggingFace model ID — xác định LLM cần tải (có thể tham chiếu mô hình lưu trong MinIO hoặc HuggingFace Hub)
- Các tham số lệnh vllm serve — người dùng cấu hình các thông số phục vụ chính
- Số replica — số lượng bản sao Deployment vLLM (mặc định: 1)
- Port được phơi ra ngoài - dựa trên Node Port Kubernetes
- API Key

Khi pod vLLM engine đã chạy, một lệnh curl mô tả API endpoint sẽ được gửi đến người dùng.